

# Accelerating Scientific AI for Drug Discovery: OpenFold Optimization with oneDNN

**Speakers:** Rakshith G B, Shreyas Shankar  
Fujitsu Research of India Private Limited

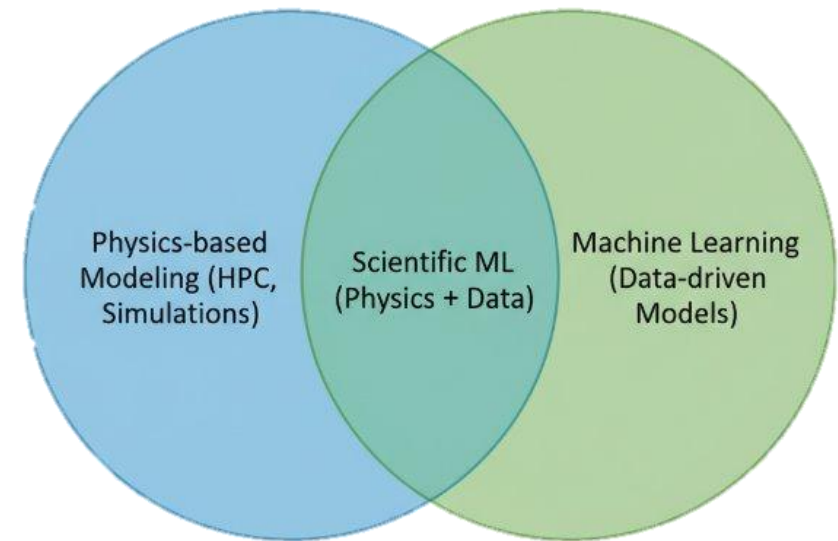


# Contents

- Introduction to Protein Folding and OpenFold
- Soloseq and MSA pipelines in Drug Discovery
- Role of JIT BRGEMM Kernel performance in OpenFold
- Benchmark Results of OpenFold on ARM CPU
- Concluding Remarks and Resources

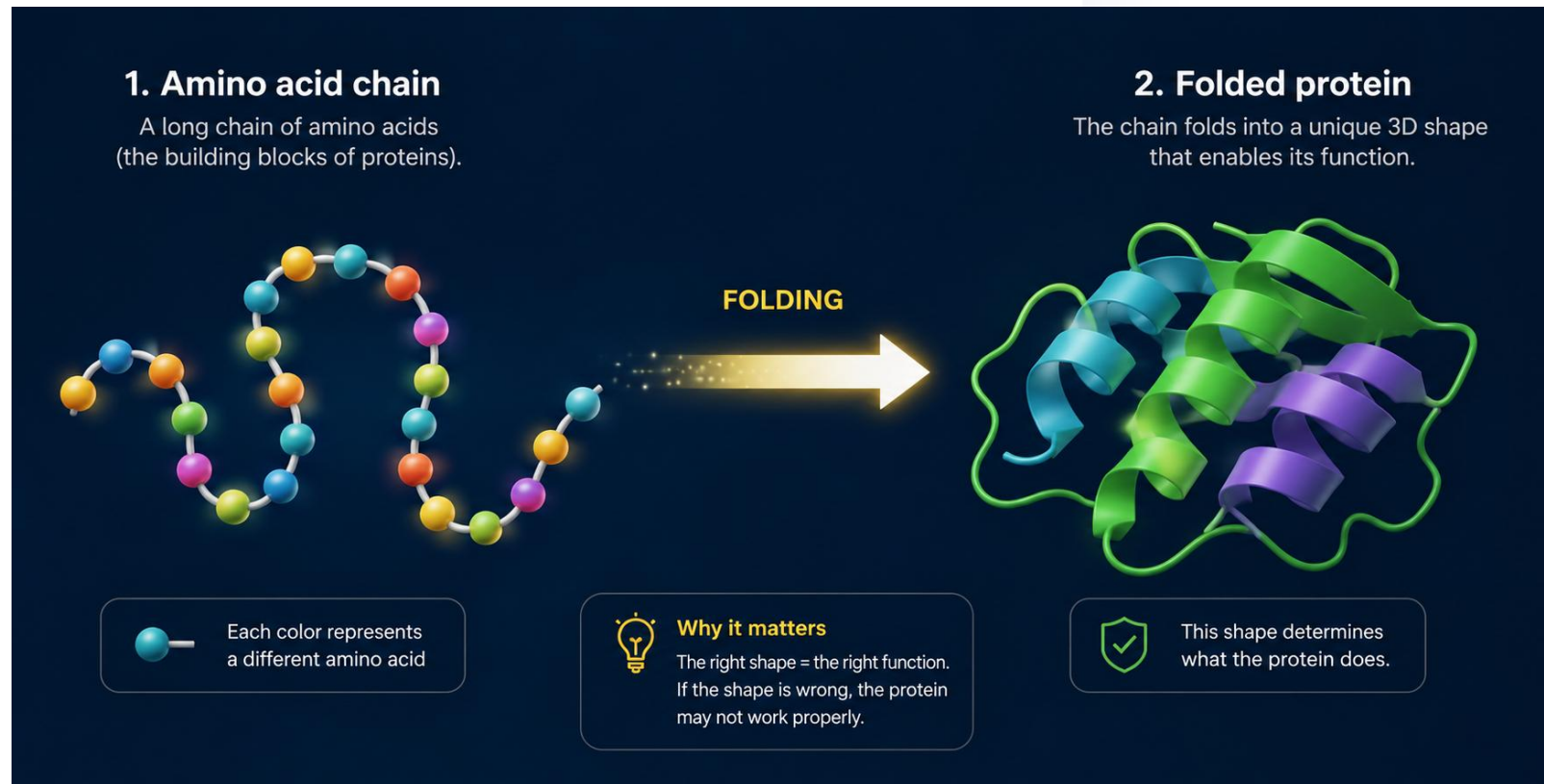
# What is Scientific Computing

- Scientific domains such as physics, chemistry, and biology rely on mathematical equations to model real-world phenomena.
- Scientific Machine Learning (SciML) combines ML with scientific knowledge and governing equations to accelerate simulations, improve predictions, and study complex systems that are difficult or costly to model using traditional methods.



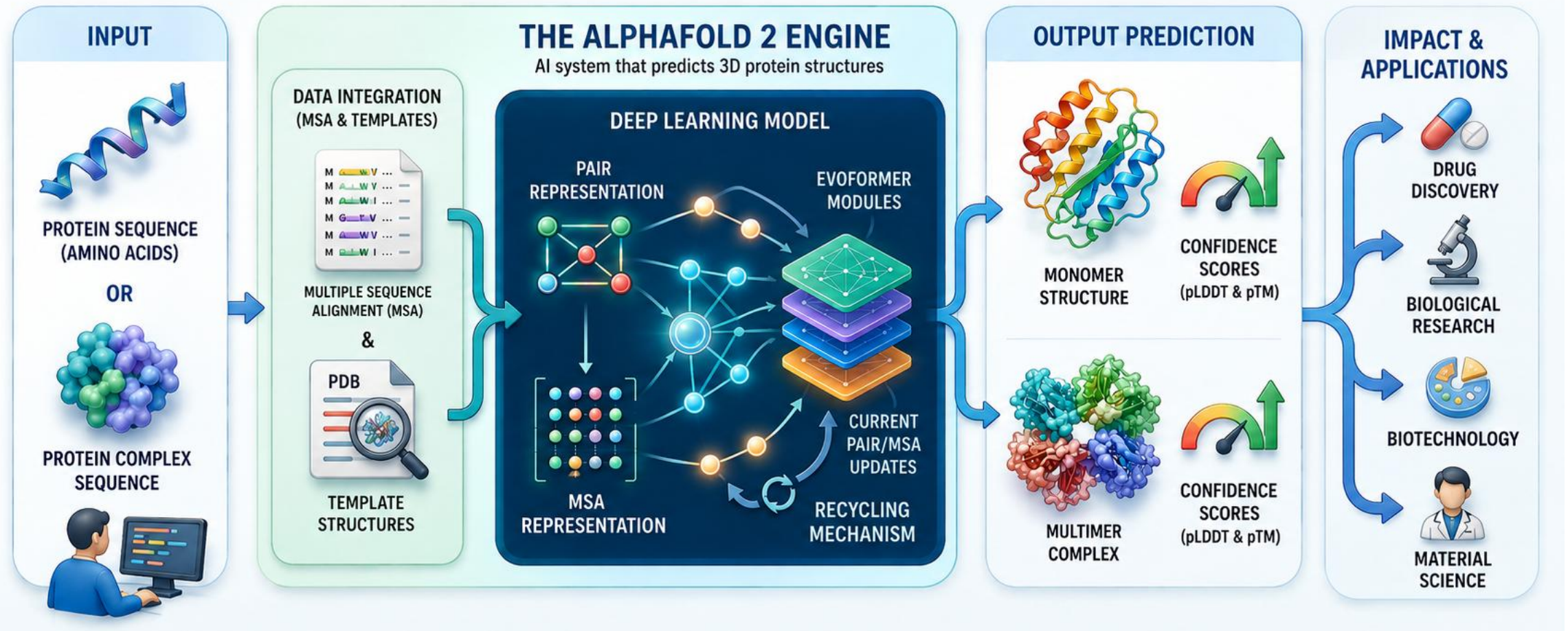
# What is Protein Folding?

- Proteins are chains of amino acids that fold into precise 3D shapes. Their shape determine their function.
- The Challenge:** A protein with 100 amino acids has  $\sim 10^{300}$  possible conformations. Experimentally determining structure (X-Ray crystallography) takes months to years and costs thousands of dollars per protein.



# What is OpenFold?

An open-source implementation of the protein structure prediction system developed to reproduce the capabilities of AlphaFold.



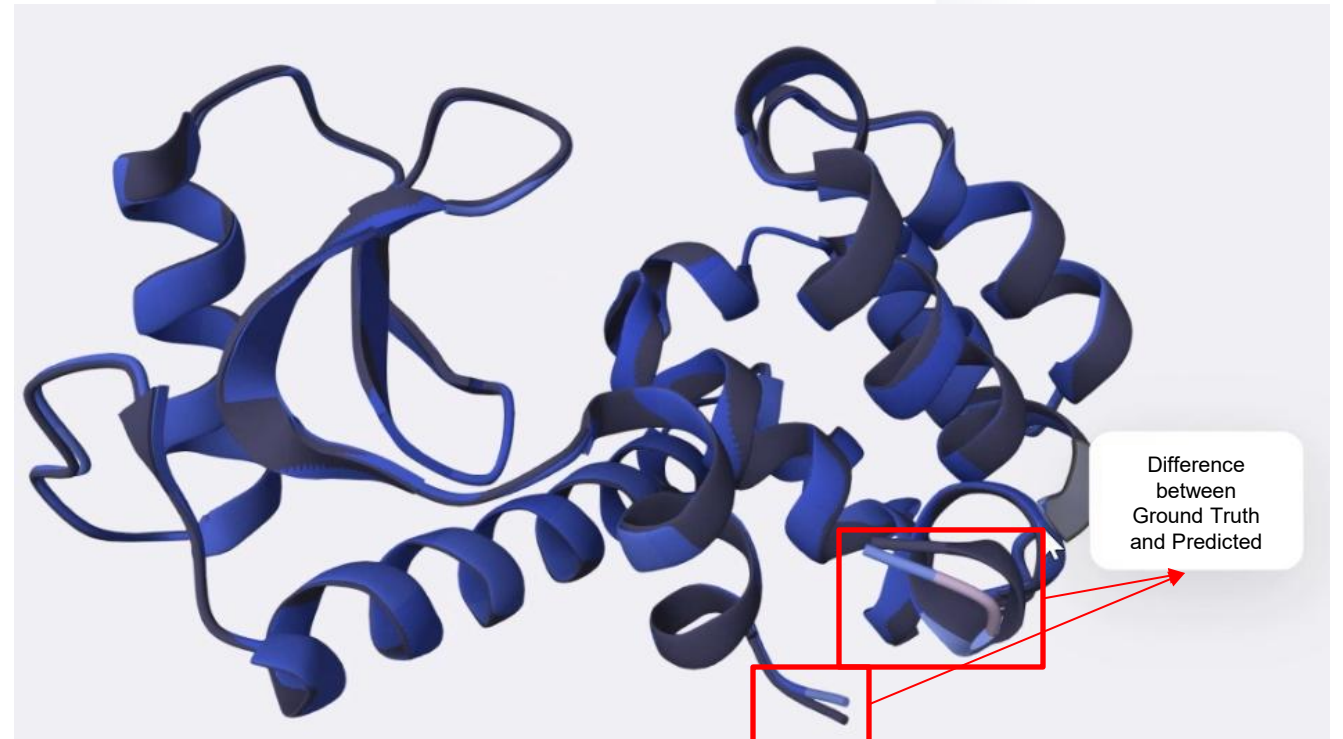


# OpenFold Evaluation & Accuracy

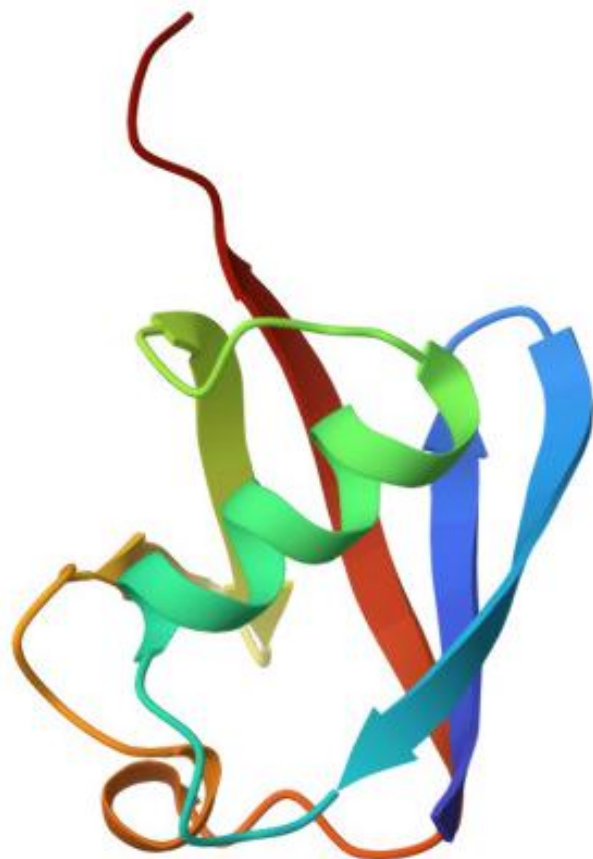
- OpenFold's Accuracy is evaluated using the standard pLDDT (predicted local distance difference test) score which is a confidence score that predicts local accuracy and stereochemistry.
- The drug discovery community prefers the Ca RMSD score which calculates the average distance of Ca atoms between a predicted structure and a known ground truth. This value if it is below 2Å is considered a good prediction.
- This is calculated using –

$$\text{RMSD} = \sqrt{\frac{1}{N} \sum \delta_i^2}$$

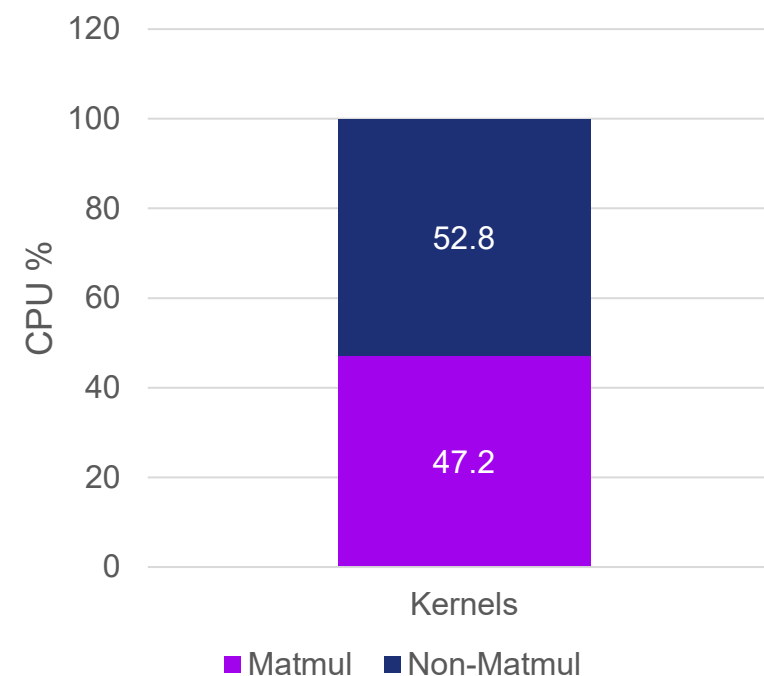
$N = \text{Total Atoms}$   
 $\delta_i = \text{Deviation per atom}$



# OpenFold Performance Bench



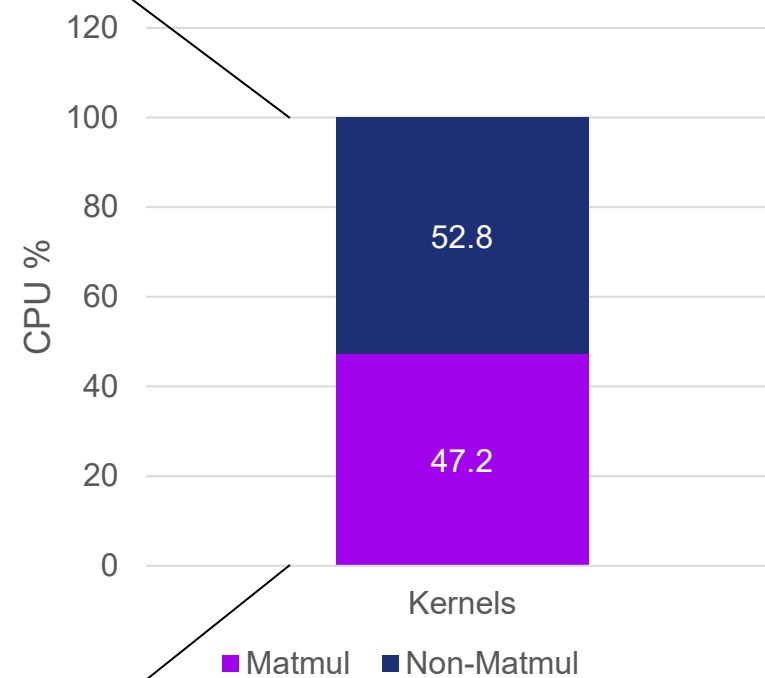
- 1UBQ (**76** residues in length)
- Took about **7.3s** on ARM CPU with 32 vCPUs.
- Accuracy of **0.24Å**.



# JIT BRGEMM in OpenFold

| #  | op                      | self (ms) | % of total | calls  | Backend / kernel                   |
|----|-------------------------|-----------|------------|--------|------------------------------------|
| 1  | aten::addmm             | 4182.3    | 28.44%     | 8000   | <b>oneDNN brg:sve_256 (BRGEMM)</b> |
| 2  | aten::copy_             | 2671.4    | 18.17%     | 25094  | memcpy / layout                    |
| 3  | aten::mm                | 1273.5    | 8.66%      | 2884   | Eigen/OpenBLAS (not oneDNN)        |
| 4  | aten::bmm               | 1257.6    | 8.55%      | 2980   | <b>oneDNN brg:sve_256 (BRGEMM)</b> |
| 5  | aten::native_layer_norm | 847.8     | 5.76%      | 4750   | native                             |
| 6  | aten::mul_              | 619.3     | 4.21%      | 4416   | native (eltwise)                   |
| 7  | aten::add_              | 615.8     | 4.19%      | 3692   | native (eltwise)                   |
| 8  | aten::sigmoid_          | 389.7     | 2.65%      | 2290   | native (eltwise)                   |
| 9  | aten::fill_             | 280.2     | 1.91%      | 62172  | native                             |
| 10 | aten::sum               | 221.3     | 1.50%      | 29902  | native (reduction)                 |
| 11 | aten::mul               | 218.7     | 1.49%      | 11416  | native (eltwise)                   |
| 12 | aten::div_              | 199.1     | 1.35%      | 30120  | native (eltwise)                   |
| 13 | aten::_softmax          | 185.1     | 1.26%      | 746    | native                             |
| 14 | aten::as_strided        | 144.7     | 0.98%      | 186740 | view/meta                          |
| 15 | aten::select            | 133.0     | 0.90%      | 64987  | view/meta                          |

JIT BRGEMM occupies ~37% of the total runtime

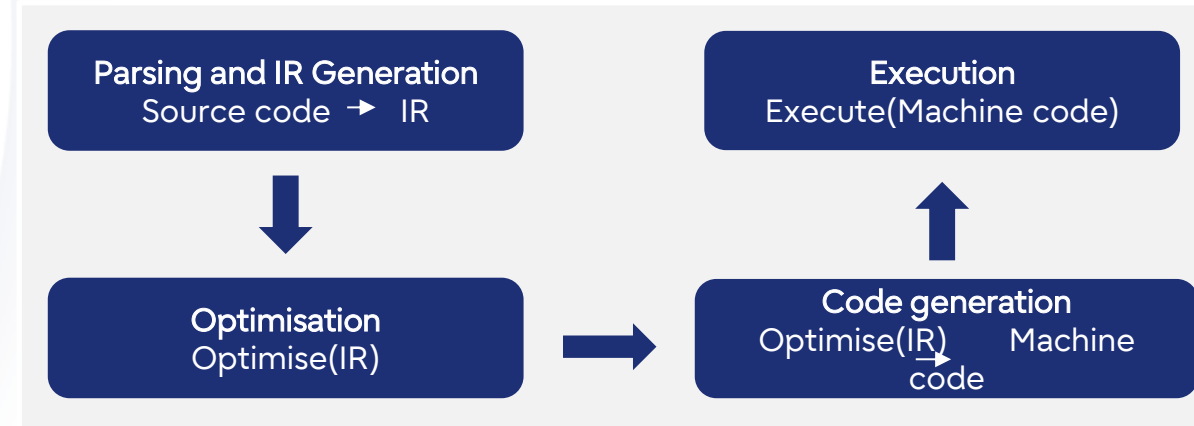




# JIT Kernels for Arm CPUs

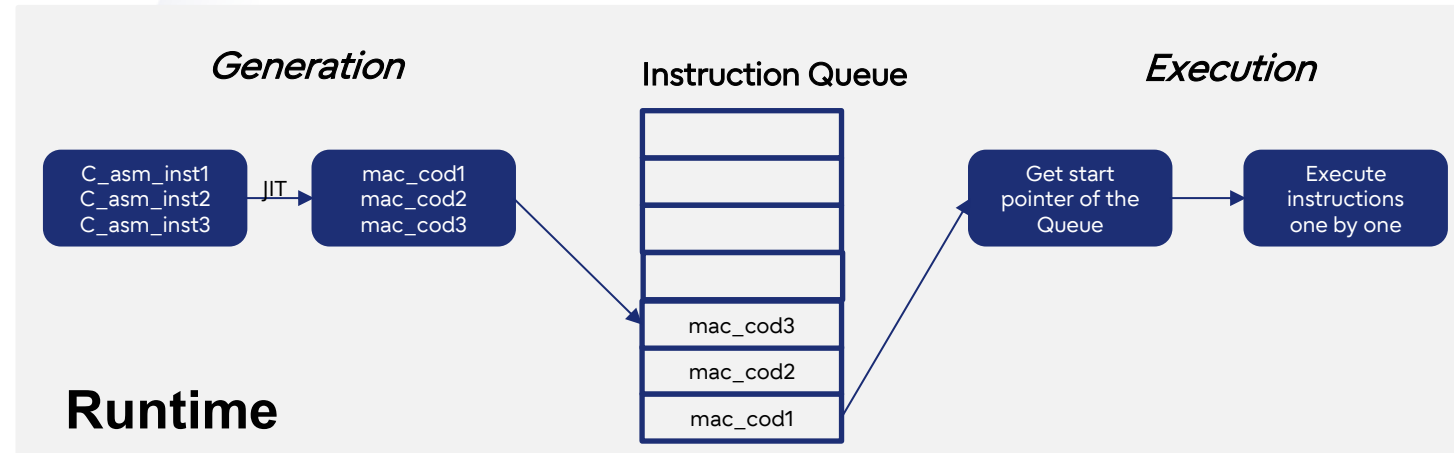
- **JIT Compiler**

- Just-In-Time (JIT) compilation is a technique used in compiler design where the compiler translates source code into machine code **at runtime**, rather than ahead of time (AOT) as in traditional compilation.



- **Xbyak aarch\_64**

- Xbyak\_aarch64 is a C++ library which enables run-time assemble coding with the AArch64 instruction set of Arm(R)v8-A architecture.
- This JIT compiler goes through 2 phases, code generation phase and code execution phase.



# Advantages of JIT Kernels for Arm CPUs

## Specialized Code Generation

**Specialized code generation:**  
Generating specialized code paths based on runtime type information Ex. specific input data or hardware capabilities.

## Dead Code Elimination

**Dead code elimination:**  
Identifying and removing code that does not contribute  
Ex. unreachable code.

## Loop Optimization

**Loop optimizations:** Loop  
Unrolling, fusion,  
vectorisation.

## Register Allocation

**Register allocation:** Assigning  
variables to processor  
registers to minimize memory  
access and improve  
performance.

## Code Reordering

**Code Reordering:**  
Instruction scheduling &  
Memory access  
optimization

### SVE Intrinsics

```
if(input == 4)
{
    sva=svld1_f32(p0,addr1)
    svb=svld1_f32(p0,addr2)
}
sv=svld1_f32(p0,addr)
```

After compilation (Irrespective of input)

```
cmp(input, 4)
b(NE, 10)
ld1w(z0, p0, addr1)
ld1w(z1, p0, addr2)
Label(10)
ld1w(z3, p0, addr)
```

### JIT

```
if(input == 4)
{
    ld1w(z0, p0, addr1)
    ld1w(z1, p0, addr2)
}
ld1w(z3, p0, addr)
```

After compilation(Input = 4)

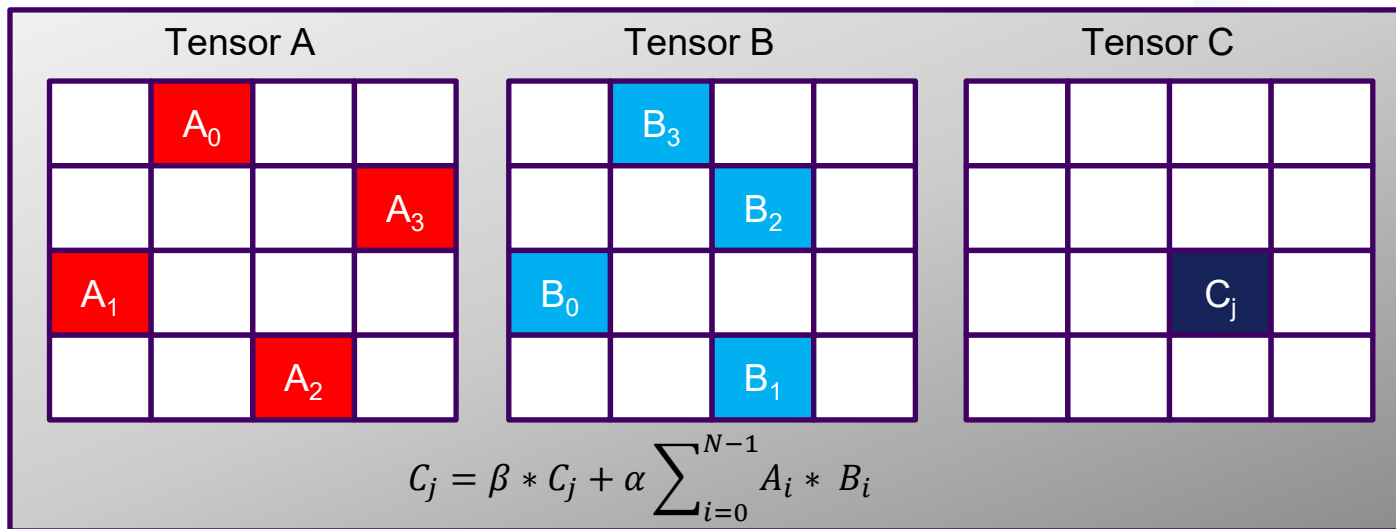
```
ld1w(z0, p0, addr1)
ld1w(z1, p0, addr2)
ld1w(z3, p0, addr)
```

After compilation(Input = 5)

```
ld1w(z3, p0, addr)
```

# BRGEMM on Arm (FP32) - Enablement

**BRGEMM** : The batch-reduce GEMM kernel performs multiplication on a series of input sub-tensor blocks (batch) and combines the partial results into a single output or accumulator sub-tensor block.<sup>[1]</sup>

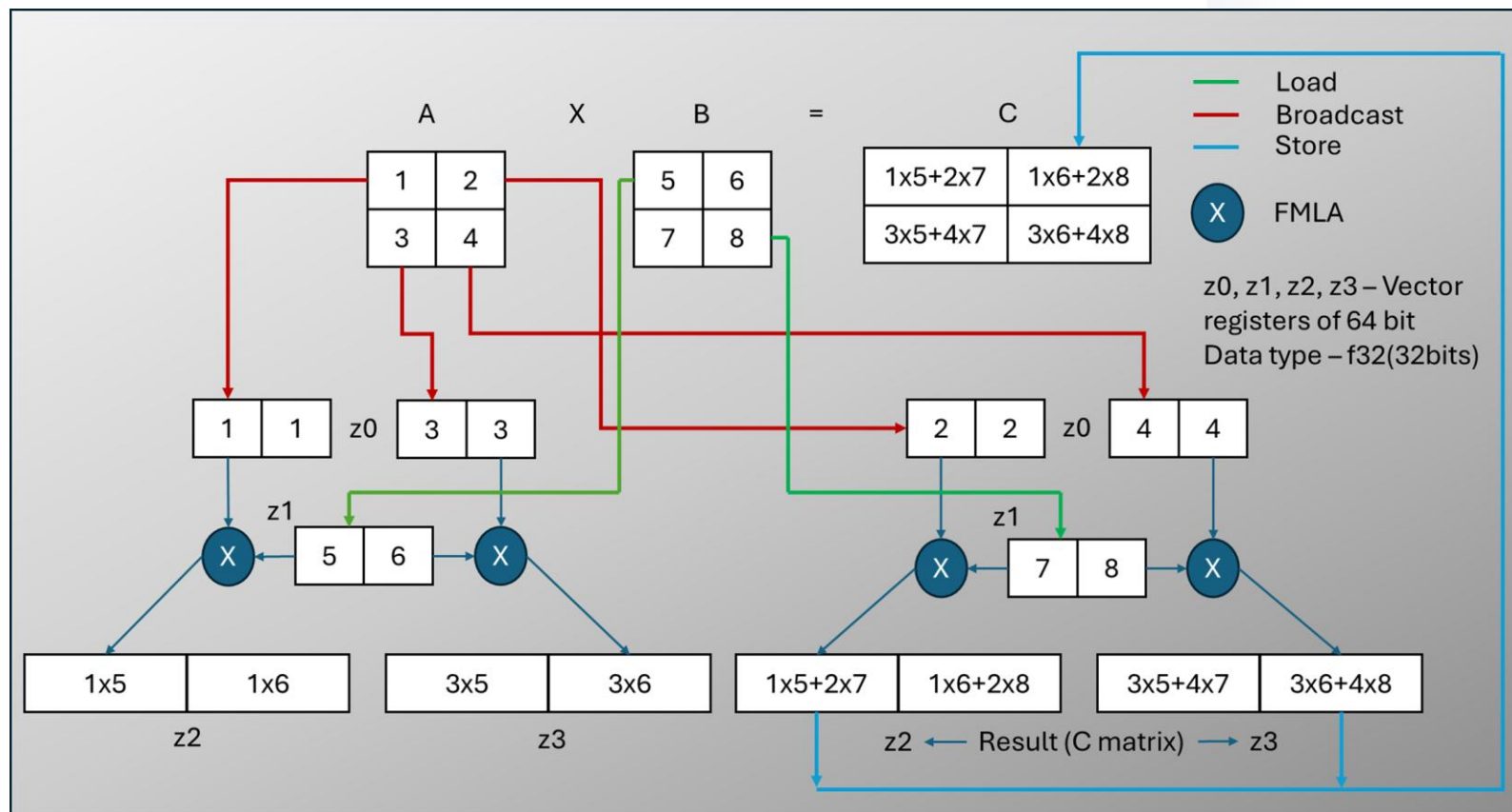


*Batch-reduce operation*

*The 4 sub-tensors of tensor A are multiplied with the corresponding 4 sub-tensors of B forming a batch and reduced to a sub-tensor of C as show in the formula, where  $\alpha$  and  $\beta$  are scaling parameters. This reduces load/store operations on C tensor<sup>[1]</sup>.*

4 matrix multiplications are performed in a batch, i.e.  $A_0 \times B_0$ ,  $A_1 \times B_1$ ,  $A_2 \times B_2$ ,  $A_3 \times B_3$  and the sum of all the results is store in  $C_j$ .

# GEMM Microkernel of BRGEMM

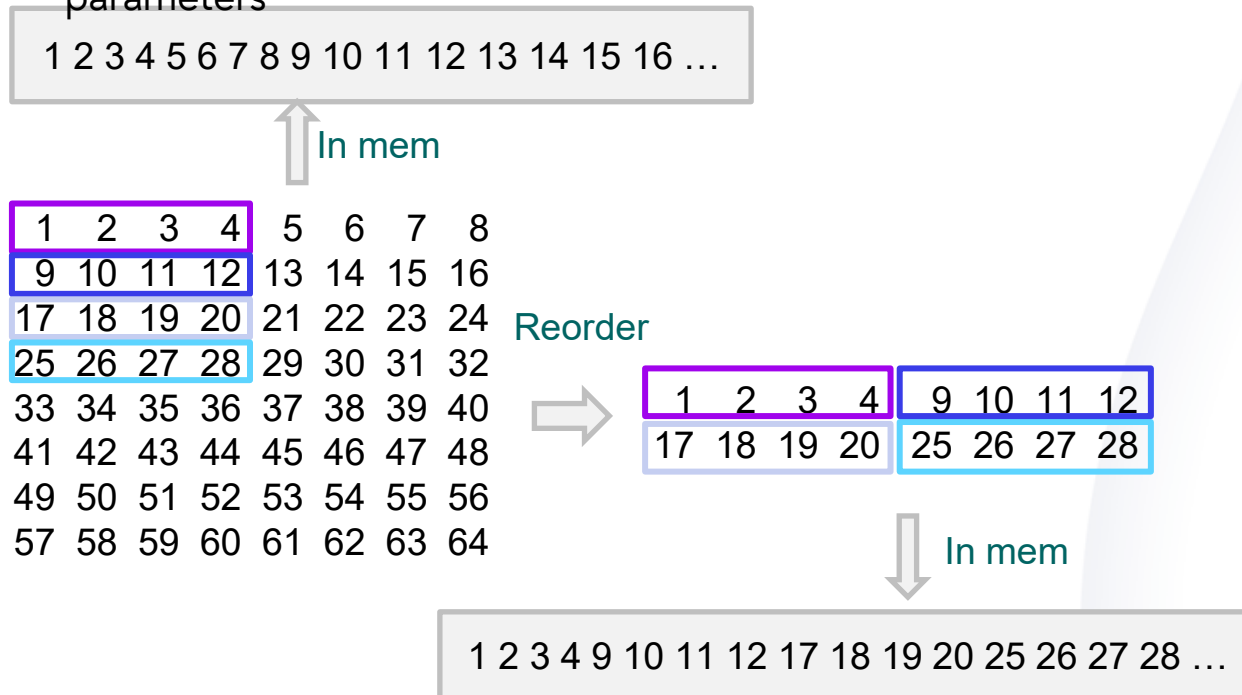


The GEMM microkernel implementation used by BRGEMM on aarch64 for a single matrix multiplication (e.g. -  $A_0 \times B_0$ ) within a batch, written in JIT using Xbyak library in C++. Here for simplicity each vector registers is considered to be 64 bits, which means each vector register can hold two f32 elements.

# BRGEMM on Arm (FP32) – Tuning Optimizations

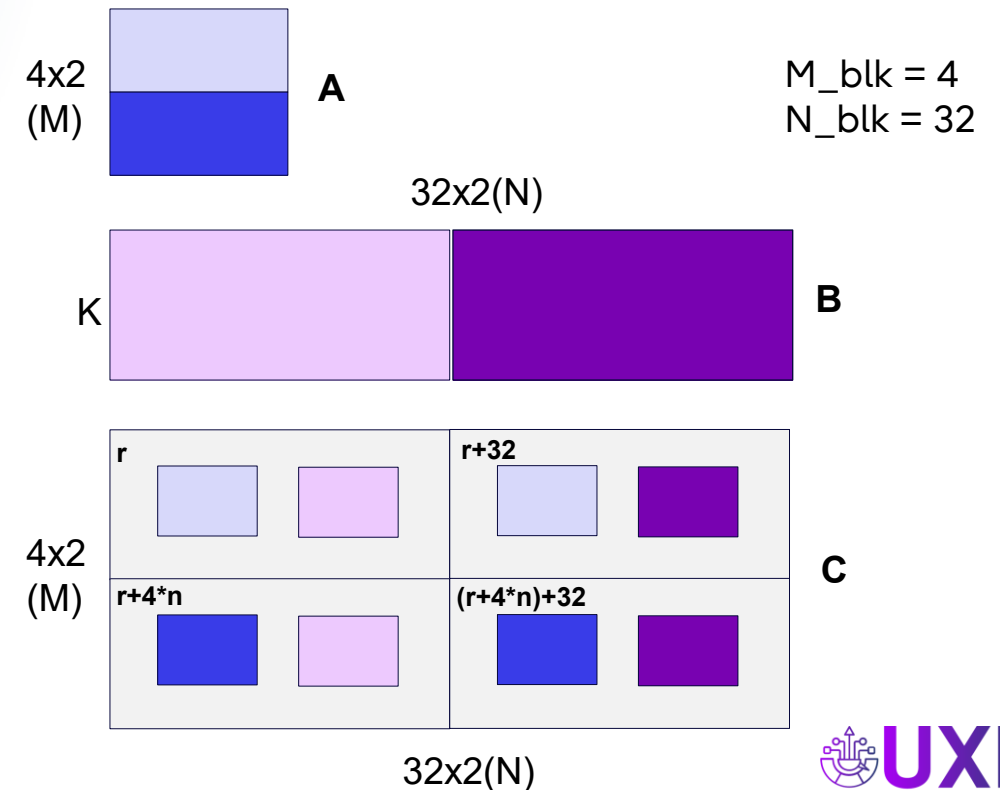
## 1# Reorder

- Data reordering groups together memory blocks that are accessed sequentially, minimizing cache misses and reducing memory latency.
- In BRGEMM, we reorder the weight matrix based on blocking parameters



## 2# Blocking

We block M and N according to input shapes, available threads, and vector registers to achieve the most optimized performance.



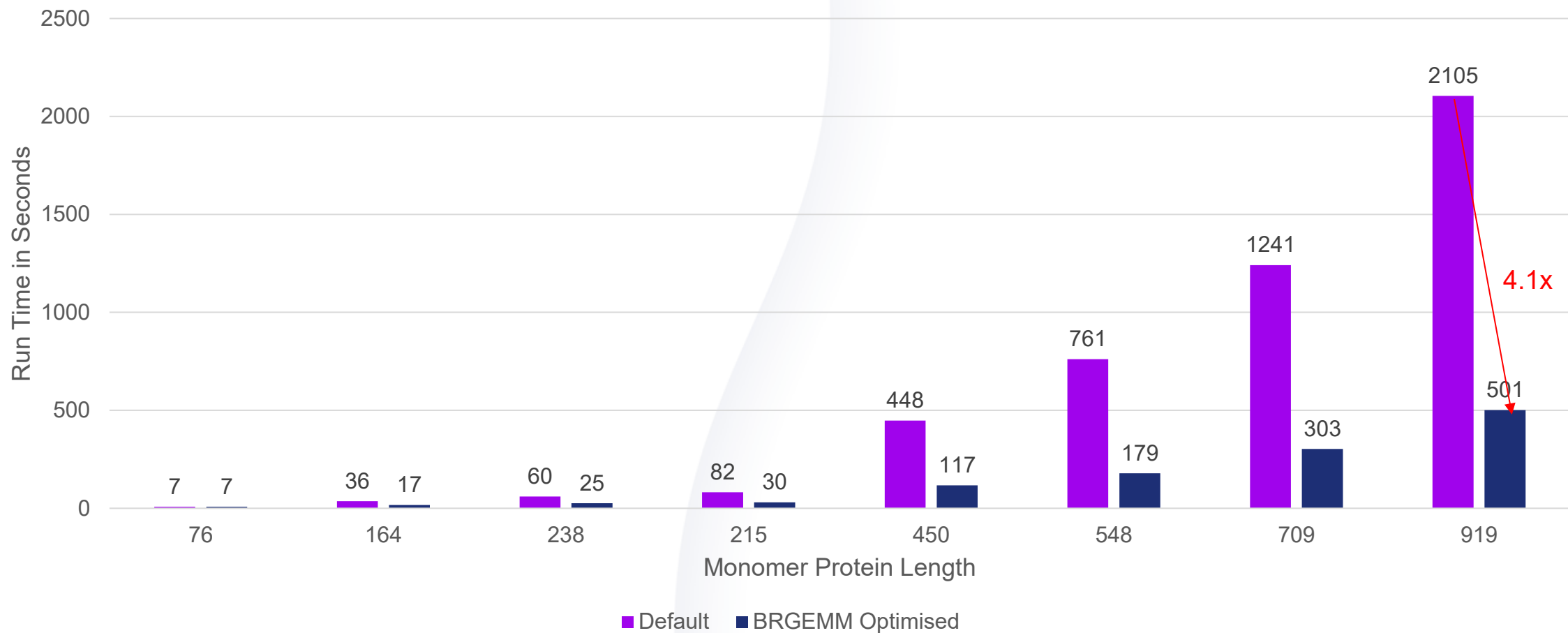


# BRGEMM OSS Contributions

Fujitsu has contributed multiple OSS contributions in oneDNN towards AI Frameworks Enablement and Performance Tuning on ARM CPUs. Below list includes few PRs supporting JIT BRGEMM and Matmul tuning on ARM.

| S.no | Description                        | PR Link                                                                                                           |
|------|------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 1    | BRGEMM FP32 for SVE 512            | <a href="https://github.com/uxlfoundation/oneDNN/pull/1867">https://github.com/uxlfoundation/oneDNN/pull/1867</a> |
| 2    | BRGEMM FP32 for SVE 256            | <a href="https://github.com/uxlfoundation/oneDNN/pull/1899">https://github.com/uxlfoundation/oneDNN/pull/1899</a> |
| 3    | BRGEMM FP32 thread optimisation    | <a href="https://github.com/uxlfoundation/oneDNN/pull/2103">https://github.com/uxlfoundation/oneDNN/pull/2103</a> |
| 4    | Matmul INT8                        | <a href="https://github.com/uxlfoundation/oneDNN/pull/2686">https://github.com/uxlfoundation/oneDNN/pull/2686</a> |
| 5    | Matmul INT8 mem/thr optimisation   | <a href="https://github.com/uxlfoundation/oneDNN/pull/3348">https://github.com/uxlfoundation/oneDNN/pull/3348</a> |
| 6    | BRGEMM FP32 Convolution SVE512     | <a href="https://github.com/uxlfoundation/oneDNN/pull/1968">https://github.com/uxlfoundation/oneDNN/pull/1968</a> |
| 7    | BRGEMM FP32 Convolution SVE256     | <a href="https://github.com/uxlfoundation/oneDNN/pull/1983">https://github.com/uxlfoundation/oneDNN/pull/1983</a> |
| 8    | BRDGEMM FP32 Depthwise Convolution | <a href="https://github.com/uxlfoundation/oneDNN/pull/2009">https://github.com/uxlfoundation/oneDNN/pull/2009</a> |
| 9    | BRGEMM INT8 Kernel                 | <a href="https://github.com/uxlfoundation/oneDNN/pull/3414">https://github.com/uxlfoundation/oneDNN/pull/3414</a> |
| 10   | BRGEMM INT8 Convolution            | <a href="https://github.com/uxlfoundation/oneDNN/pull/3552">https://github.com/uxlfoundation/oneDNN/pull/3552</a> |

# OpenFold Benchmark Results with oneDNN



# Conclusion

- Fujitsu continues to actively work on scientific ML and AI surrogate development.
- As part of this work we enhance model architectures to improve accuracy of AI vs HPC and performance tuning utilising and contributing to all major AI frameworks (oneDNN, pytorch etc.)
- All this work is towards building optimised software stack for FUJITSU-MONAKA processor which is on track for launch in FY27.

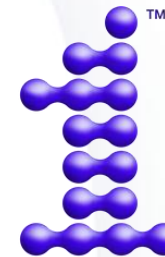
# Resources

- [1] Georganas, Evangelos & Banerjee, Kunal & Kalamkar, Dhiraj & Avancha, Sasikanth & Venkat, Anand & Anderson, Michael & Henry, Greg & Pabst, Hans & Heinecke, Alexander. (2019). High-Performance Deep Learning via a Single Building Block. 10.48550/arXiv.1906.06440.
- [2] Ahdritz, G., Bouatta, N., Kadyan, S., Xia, Q., Gerecke, W., O'Donnell, T. J., Berenberg, D., Fisk, I., Zanichelli, N., Zhang, B., Nowaczynski, A., Wang, B., Stepniewska-Dziubinska, M. M., Zhang, S., Ojewole, A., Guney, M. E., Biderman, S., Watkins, A. M., Ra, S., Lorenzo, P. R., Nivon, L., Weitzner, B., Ban, Y., Ban, Y. A., Sorger, P. K., Mostaque, E., Zhang, Z., Bonneau, R., & AlQuraishi, M. OpenFold: Retraining AlphaFold2 yields new insights into its learning mechanisms and capacity for generalization. <https://doi.org/10.1101/2022.11.20.517210>
- [3] FUJITSU-MONAKA Introduction - <https://global.fujitsu/en-global/technology/research/fujitsu-monaka>
- [4] Fujitsu's AI Surrogate Model Platform - <https://en-documents.research.global.fujitsu.com/ai-surrogate-model-trial-platform/>
- [5] Fujitsu's Tech Blog on FUJITSU-MONAKA <https://blog-en.fltech.dev/archive/category/FUJITSU-MONAKA>
- [6] Accelerating Scientific ML with FUJITSU-MONAKA: Towards Energy-Efficient AI for Next-Gen Simulation Workflows - [https://www.youtube.com/watch?v=ty\\_GaUMx5lA](https://www.youtube.com/watch?v=ty_GaUMx5lA)
- [7] Accelerating DL on ARM through oneDNN <https://www.youtube.com/watch?v=K4tlu-VSpi0>

# Acknowledgement

This presentation is based on results obtained from a project, JPNP21029 subsidized by the New energy and Industrial Technology Development Organization (NEDO).

# Q&A



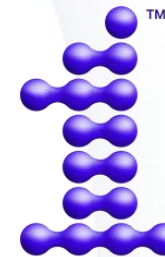
**oneAPI**



**UXL**

Unified Acceleration Foundation

# Thank You



**oneAPI**



**UXL**

Unified Acceleration Foundation